

Exercise 8

Folding

`foldl` and `foldr`

Many functions that apply to lists can be written, as we saw in lectures, in the form:

```
f [] = v
f (x:xs) = x `op` f xs
```

which turns the empty list into some initial value (here `v`) and all other lists to the result of apply a specified binary operator `op` to the head of the list and the result of recursion on the tail.

For example

```
sum :: Num a => [a] -> a
sum [] = 0
sum (x:xs) = x + sum xs

or :: [Bool] -> Bool
or [] = False
or (x:xs) = x || or xs
```

We saw that this linear recursive pattern was captured in the higher order function `foldr` (“fold right”). It is so named because untangling the recursion, we see that the operator is applied in a right-associative manner.

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

With which we have

```
sum :: Num a => [a] -> a
sum = foldr (+) 0

or :: [Bool] -> Bool
or = foldr (||) False
```

We can think of `foldr` non-recursively as replacing the “cons” operator `(:)` with the provided binary function, and the empty list with the starting value. For example

```
foldr (+) 0 [1, 2, 3, 4]
== -- expanding the sugar for the list
foldr (+) 0 (1 : (2 : (3 : (4 : []))))
== -- replacing : with + and [] with 0
(1 + (2 + (3 + (4 + 0))))
==
```

```
10
```

There is also a similar function `foldl` (“fold left”)

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

which folds from the end of the list, rather than the beginning

```
foldl (+) 0 [1, 2, 3, 4]
== -- expanding the sugar for the list
foldl (+) 0 (1 : (2 : (3 : (4 : []))))
== -- replacing : with + and [] with 0
(((1 + 2) + 3) + 4) + 0
==
10
```

This matches a pattern where the operator associates to the left, rather than the right:

```
f v [] = v
f v (x:xs) = f (v `op` x) xs
```

Which maps the empty list to an accumulator value, and any other list to the result of processing the tail using a new accumulator obtained by applying `op` to the accumulator and the head of the list.

Question 8.1

If presented with an infinite list which you wish to process in this manner, should you use `foldr` or `foldl`, or does it not matter? Explain your reasoning.

Library functions with folds

Exercise 8.2

Let’s implement a few library functions on lists by using `foldr` and `foldl` to check we understand what’s going on.

The template file for this exercise is [folds-exercise8.hs](#).

In particular, we’re going to try and implement

```
-- Length of a list
length' :: [a] -> Int
-- Map a function over a list to produce a new one
```

```
map' :: (a -> b) -> [a] -> [b]
-- Reverse a list
reverse' :: [a] -> [a]
-- Check if all entries in a boolean list are true
and' :: [Bool] -> Bool
```

Using both `foldl` and `foldr`.

Higher order functions with folds

Having done that, we'll do a few more, this time higher-order functions (again, using the same template file).

Exercise 8.3

There are three here that are relatively straightforward, and then one which is *much harder* (so don't worry if it eludes you!). Try and implement these using composition of existing higher-order functions that you already know about.

```
-- Every entry satisfies a predicate, all p [] = True
all' :: (a -> Bool) -> [a] -> Bool
-- At least one entry satisfies a predicate, any p [] = False
any' :: (a -> Bool) -> [a] -> Bool
-- Return elements from a list while they satisfy a predicate, and
-- stop as soon as the predicate is not satisfied
takeWhile' :: (a -> Bool) -> [a] -> [a]
-- Remove elements from a list while they satisfy some predicate,
-- and
-- then return the remainder. This one is MUCH HARDER.
dropWhile' :: (a -> Bool) -> [a] -> [a]
```

Fold-map fusion

You might have, and I did, implement `all'` like this

```
all' p xs = and (map p) xs
-- Or
all' p = and . map p
```

This is a nice solution, but it does traverse the list twice, once to apply `p` to every element, and then to check if they are all `True`. Fortunately, we can avoid this double traversal, using an example of provably safe program transformations. The particular one we need is *fold-map fusion* (see [Equation 11](#)).

Exercise 8.4

Try writing a function

```
allSingleFold :: (a -> Bool) -> [a] -> Bool
allSingleFold p xs = undefined
```

which does the same thing as `all` but with only one traversal over the list using `foldr`.

Think about what the operator you want to apply at every stage is.

Hint: The paper linked above shows an *equation* for fold-map fusion, namely

```
foldr op z . map p == foldr newop z
  where x `newop` y = p x `op` y
```

This is a nice property, because it means we can always switch between one form and the other, so optimising compilers can exploit the property.